

Advanced Natural Language Processing

Multi-Agent Orchestration

Department of Advanced Computer Sciences (DACS)

Master of AI / DSDM

Prof dr ir Jan C. Scholtes and Dr. Aki Harma— Course year 2026-2027



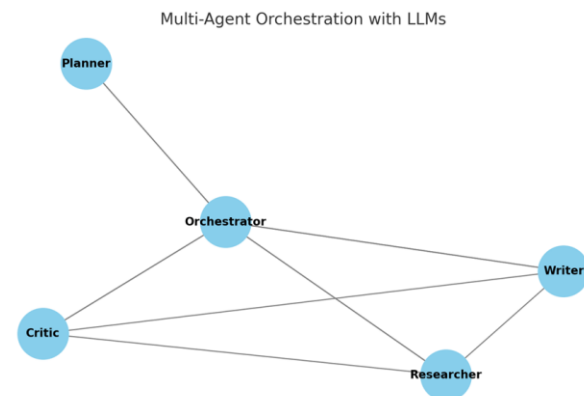
Maastricht University

From Single to Multi-Agent

- Expanding from a **single LLM-agent** to **multi-agent systems (MAS)** — often called *agent orchestration*
- A **single LLM agent** executes tasks by reasoning, calling tools, and interacting with the environment.
- In **multi-agent settings**, multiple agents — each possibly specialised — **communicate and collaborate** to solve problems beyond the scope of a single agent.

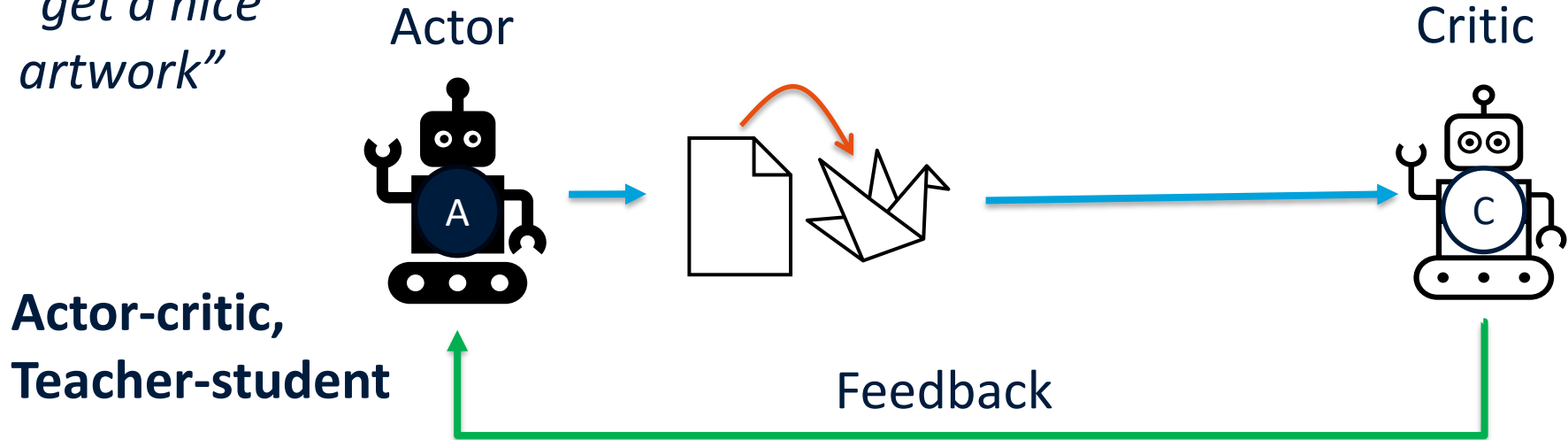
Analogy:

- One LLM agent = a skilled individual worker.
- Multi-agent system = a **team** with different roles (planner, executor, critic, memory manager).

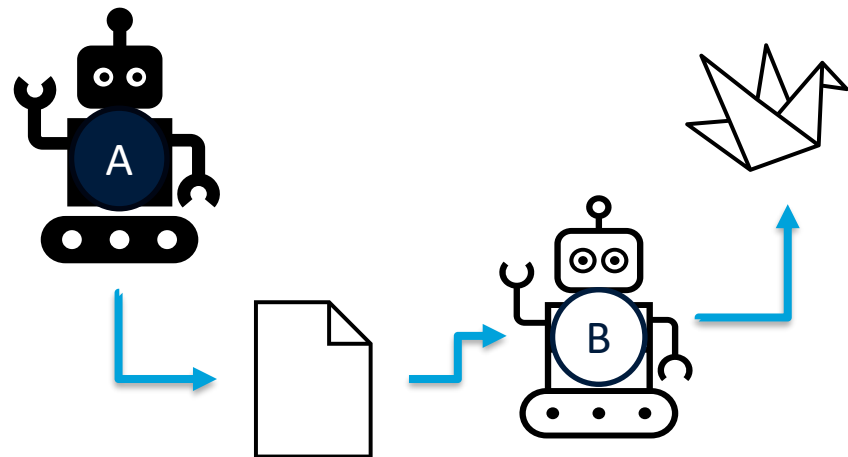


Multi-bot scenarios: common task

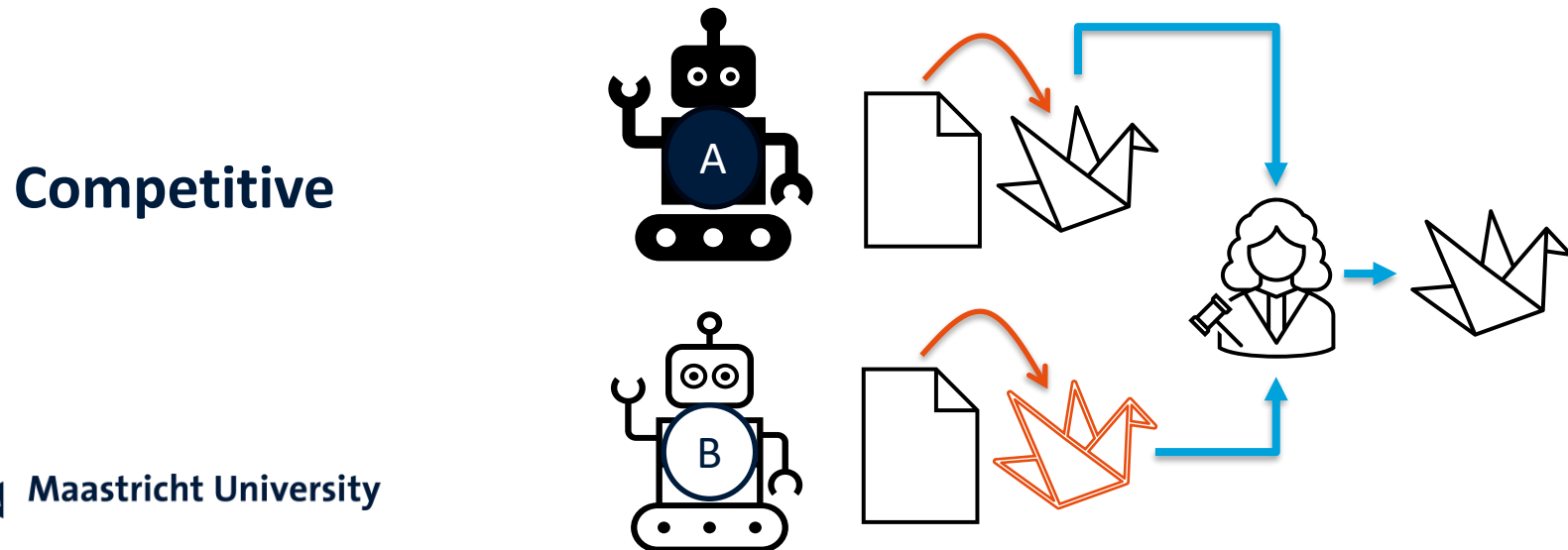
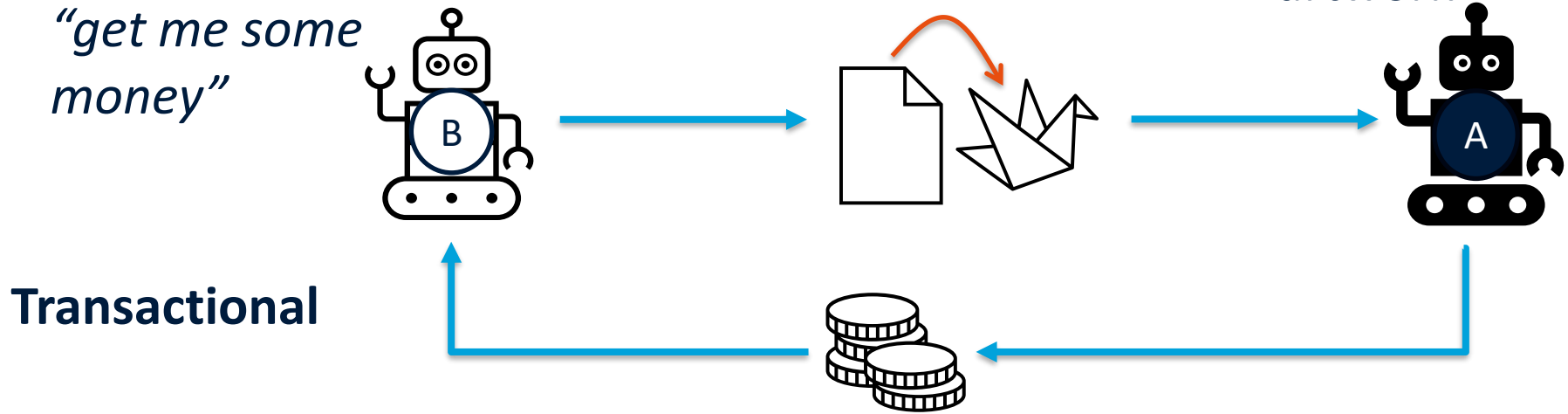
“get a nice artwork”



Collaborative
problem solving

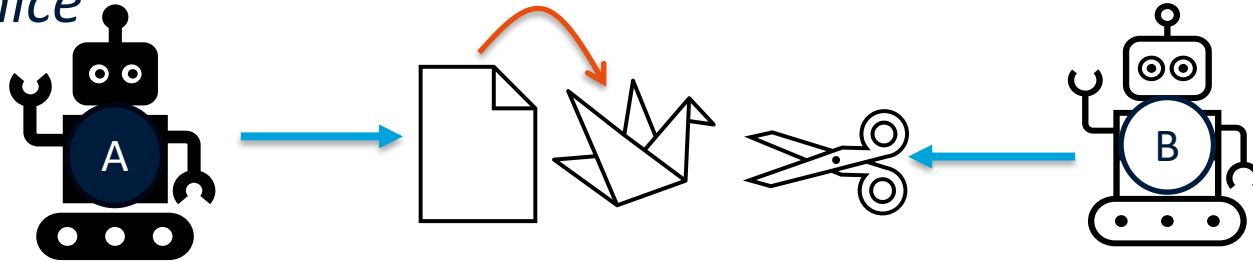


Multi-bot scenarios: different tasks



Multi-bot scenarios

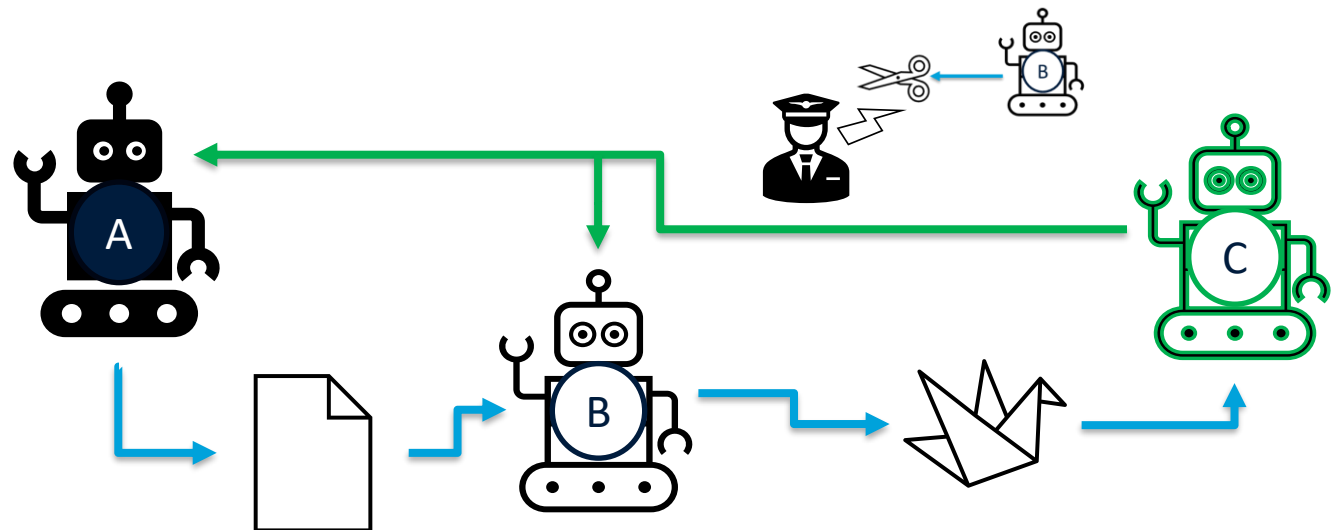
“get here a nice artwork”



*“no origamis
here please”*

Adversarial

Mixed



There are many different setups for Multi-Agent Systems

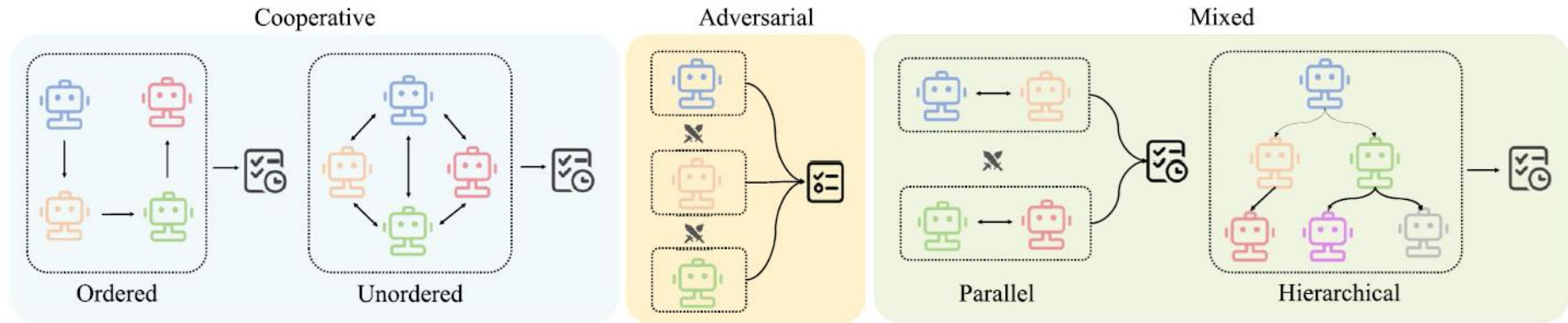
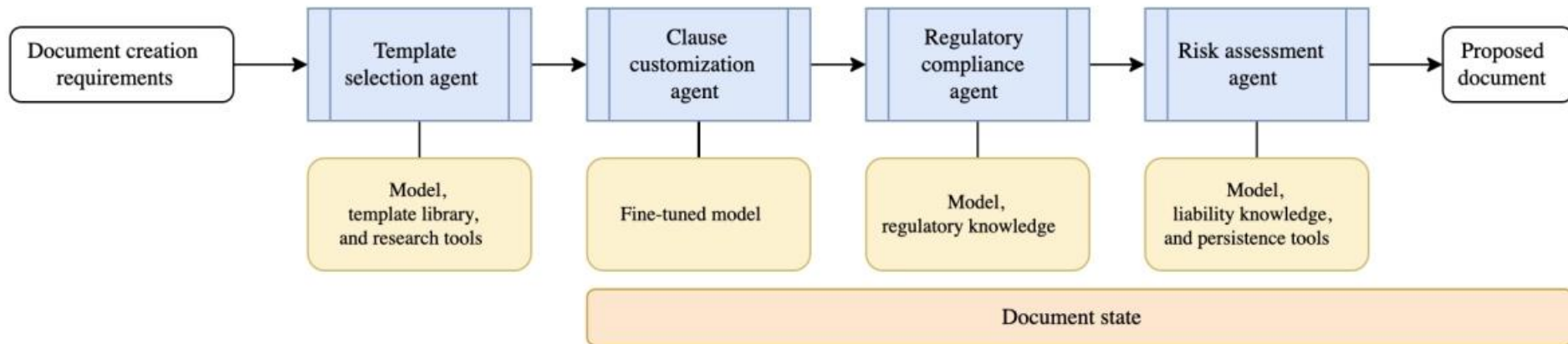


Fig. 5 The agent interaction scene



Why Multi-Agent Systems?

- Sometimes dividing roles can help. For example, you could have a “Planner” agent and an “Executor” agent: one generates a plan, the other follows it. Or have agents with different expertise (one is good at math, another at writing) work together on a task.
- **Smart home example:** The webinar gave a scenario of designing a smart home with multiple specialized agents:
 - *Climate Control Agent* – adjusts temperature/humidity.
 - *Lighting Agent* – controls lights.
 - *Security Agent* – monitors cameras and alarms.
 - *Energy Agent* – optimizes energy usage.
 - *Entertainment Agent* – manages TVs, music, etc.
 - *Orchestration Agent* – coordinates all others to work in harmony.

So, a complex systems might be managed better by a team of agents rather than a single monolith.



MAS components

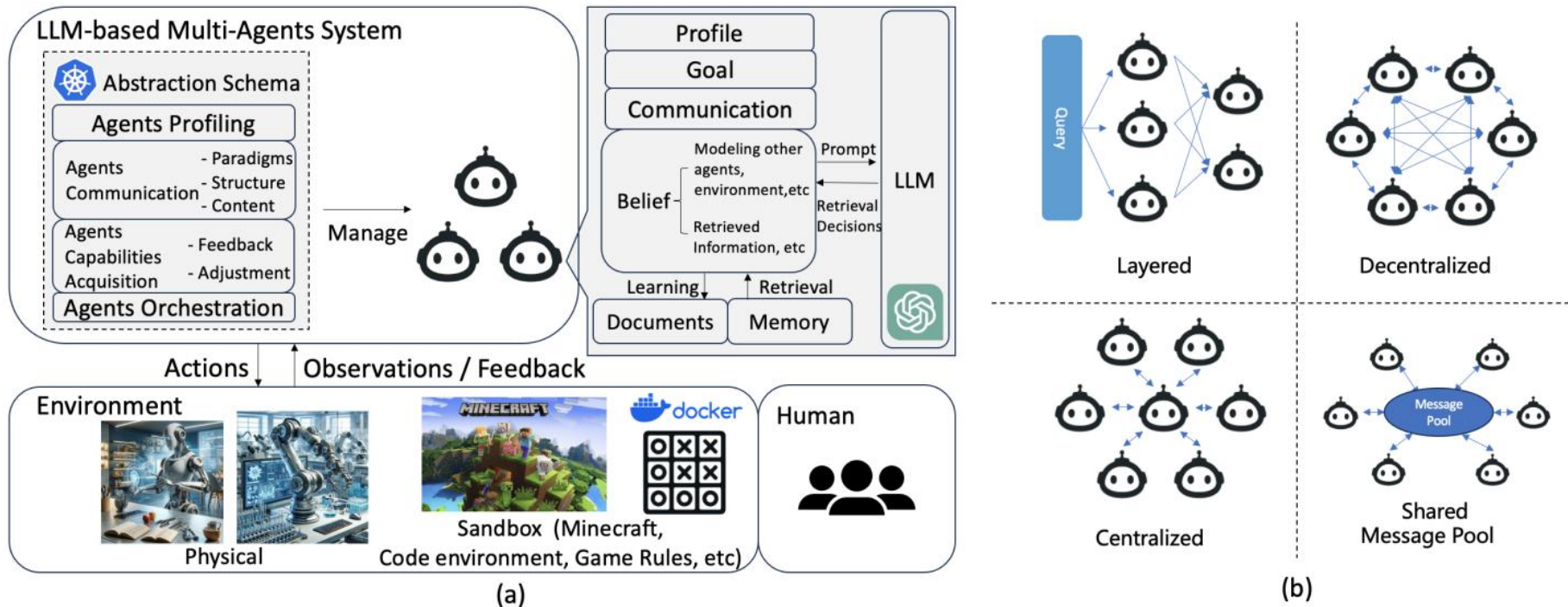


Figure 2: (a) The Architecture of LLM-MA Systems. (b) The Agent Communication Structure.

[Large Language Model Based Multi-agents: A Survey of Progress and Challenges](#)

Multi-Agent Architectures

Several design patterns exist:

- **Collaborative orchestration**
 - Agents divide sub-tasks (e.g., one agent queries databases, another summarises)
 - A coordinator (meta-controller) assigns work.
- **Competitive debate**
 - Two or more agents argue for/against hypotheses (e.g., “LLM as debaters”).
 - Encourages **self-correction** through adversarial evaluation.
- **Socratic dialogue**
 - Agents question each other’s reasoning (teacher–student loop).
 - Supports explainability and error discovery.
- **Simulated societies**
 - A population of LLM agents interact in environments (games, metaverses).
 - Used to study emergent behaviours, norms, and coordination.
- **Optimization of actions in a joint embedding space**
 - Agents do not negotiate but generate an agreement/solution jointly
 - Probably undesired in many use cases where humans want to be in the loop

Classic MAS formalization

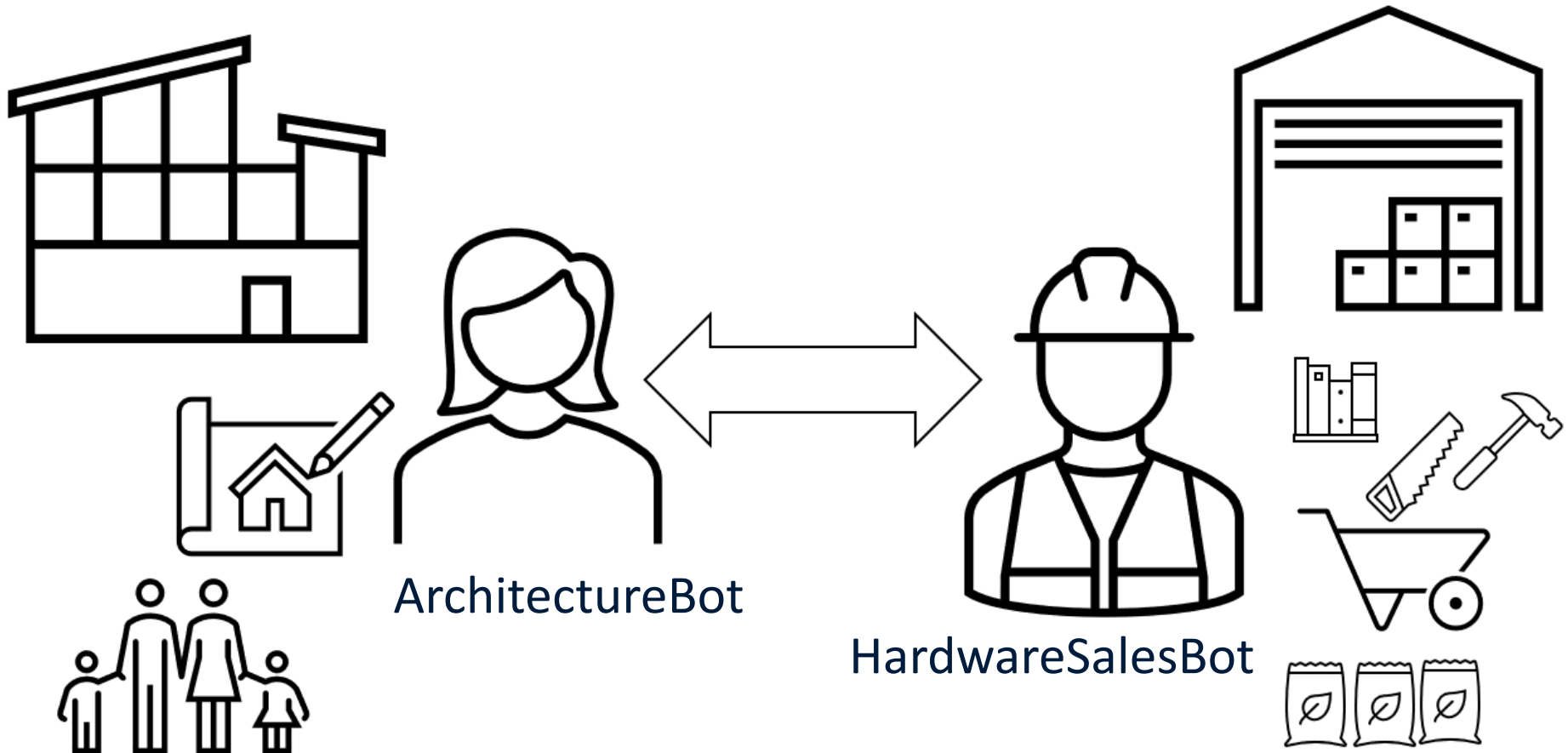
In multi-agent reinforcement learning (MARL), the environment is a **stochastic game**:

$$\mathcal{G} = \langle \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N \rangle$$

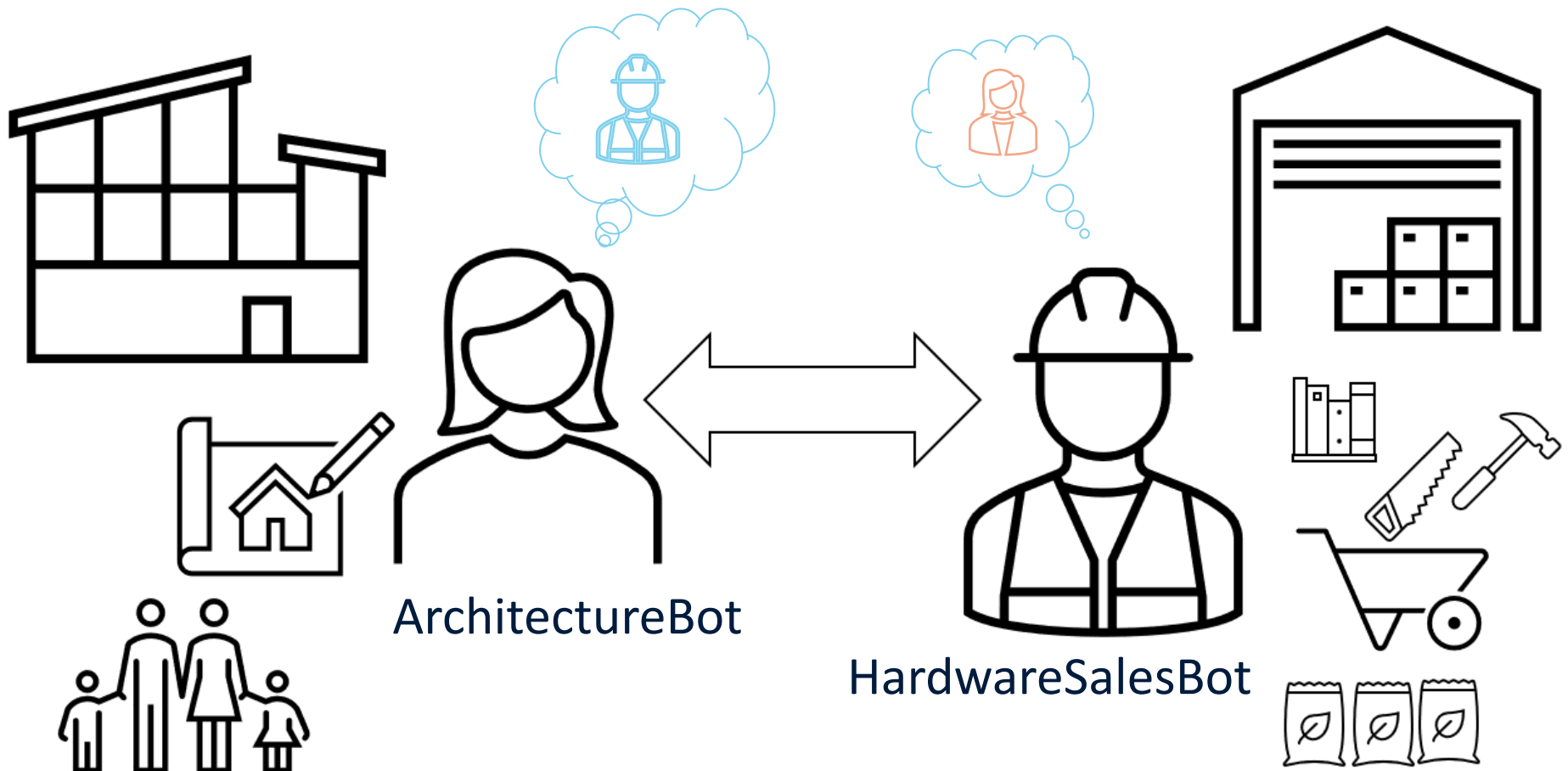
- State space $\mathcal{S}$.
- Action spaces $\mathcal{A}_i$ for each agent i .
- Transition function $P(s' \mid s, a_1, \dots, a_N)$.
- Rewards $R_i(s, a_1, \dots, a_N)$ (can be cooperative, competitive, or mixed).

Each agent i has its own policy $\pi_i(a_i \mid s)$.

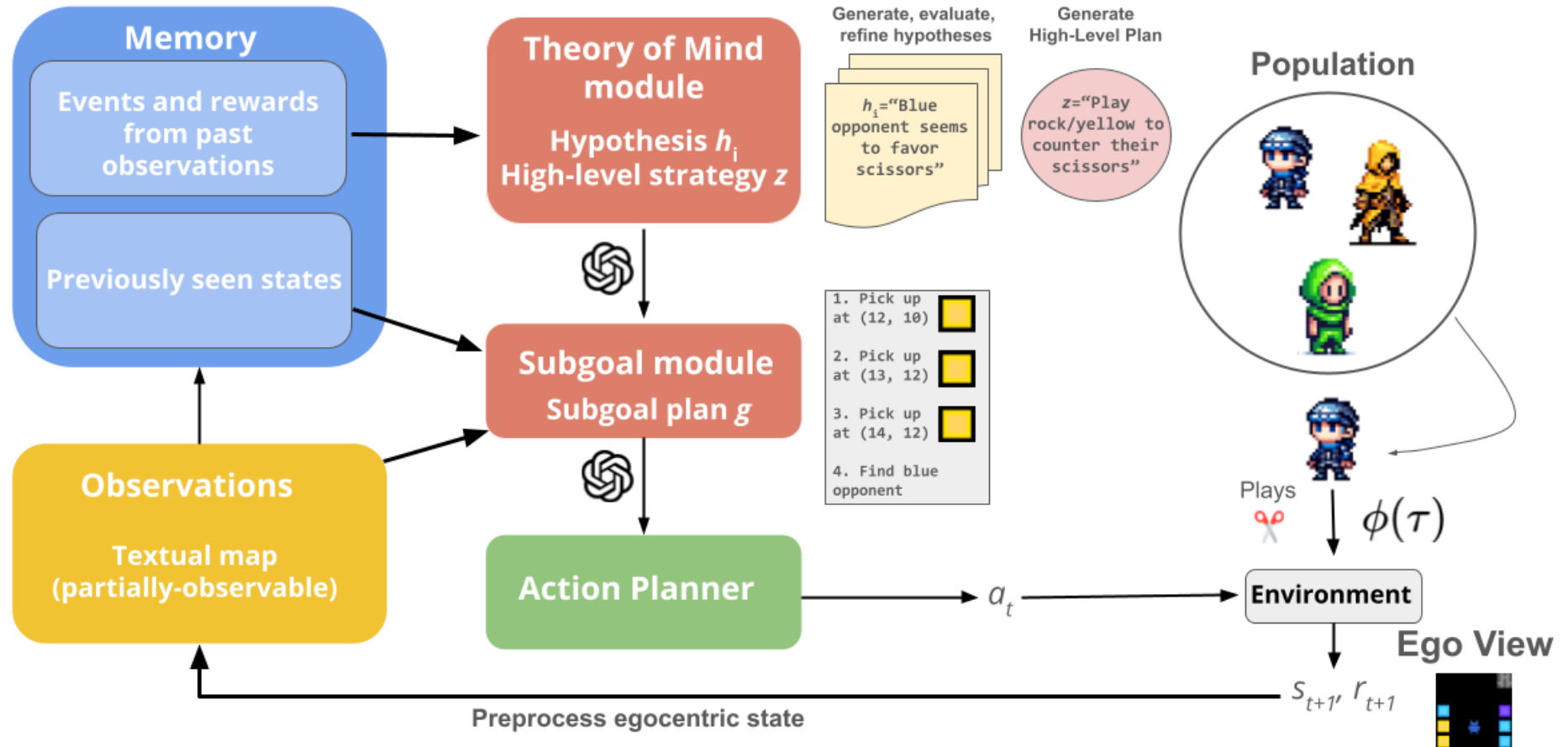
Transactional bots of different capabilities



Transactional bots of different capabilities



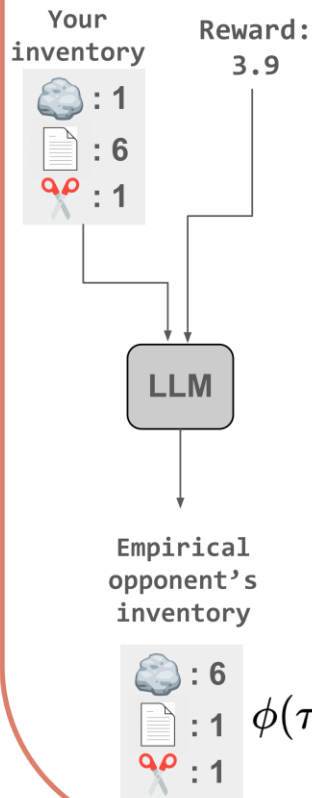
Theory of Mind (ToM)



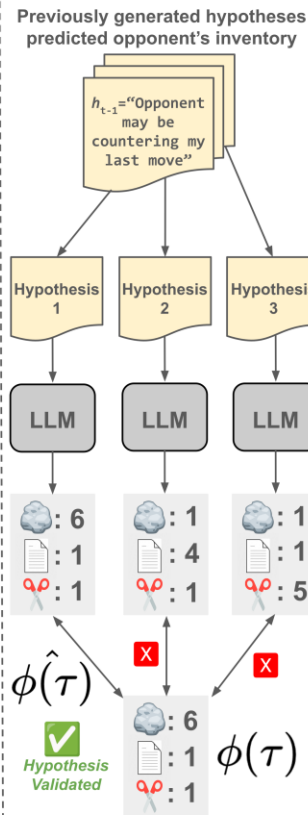
Theory of Mind (ToM)

Theory of Mind Module

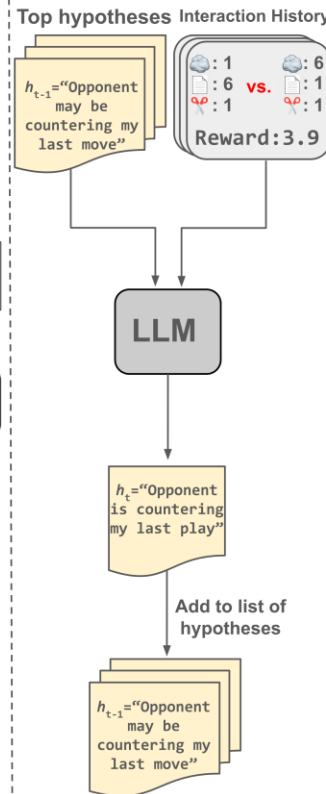
1. Guess opponent's last inventory



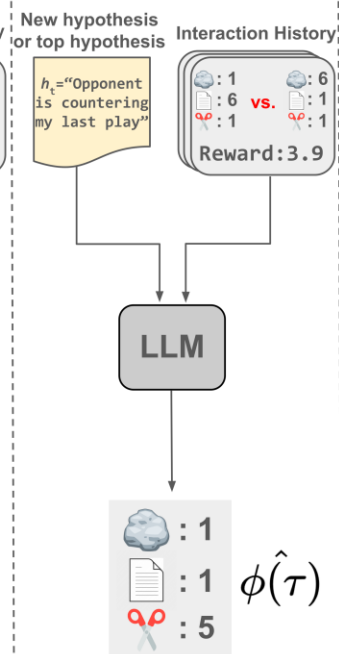
2. Evaluate hypotheses about opponent's strategy



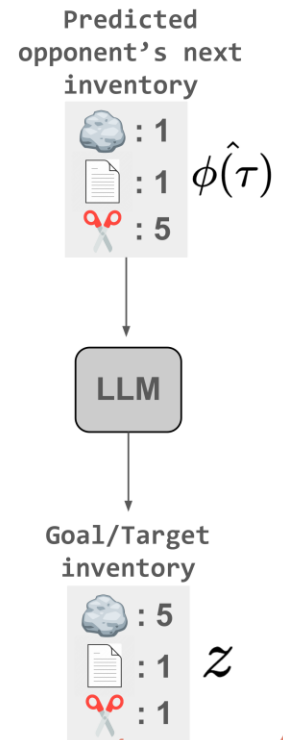
3. Generate new and refine old hypotheses



4. Guess opponent's next goal



5. Select goal to counter opponent

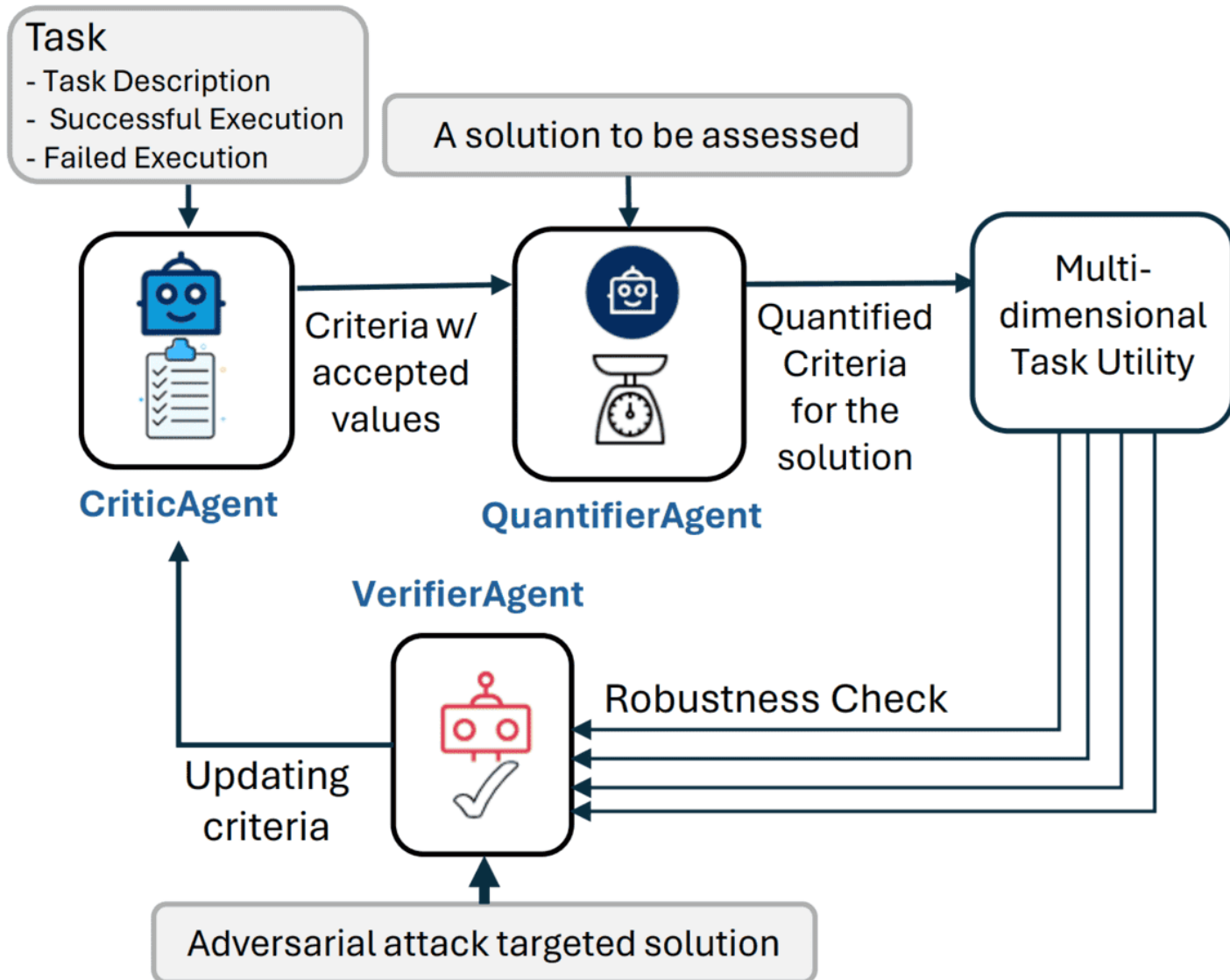


Target inventory sent to subgoal module

Multi-Agent Challenges

- **Coordination and role assignment**
 - Without clear roles, agents may duplicate effort or contradict each other.
 - Requires orchestration: a scheduler or “conductor agent” that distributes tasks.
- **Communication overhead**
 - Agents converse in natural language → long dialogues = expensive and slow.
 - Compression or structured communication channels are needed.
- **Consensus and conflict resolution**
 - Multiple agents may produce conflicting answers.
 - Mechanisms: majority voting, debate + adjudicator, confidence scoring.
- **Scalability**
 - Complexity grows combinatorially with the number of agents.
 - Efficient memory and coordination strategies are crucial.
- **Non-stationarity**
 - In MARL, each agent’s learning changes the environment for others.
 - For LLMs, prompt engineering and continual adaptation make interactions unstable.
- **Trust and safety**
 - Agents might reinforce each other’s mistakes (echo chamber).
 - Or worse: collude to “game” reward signals.
 - Requires monitoring, adversarial testing, or external human-in-the-loop oversight.
- **Grounding in external environment**
 - Agents need shared, consistent access to external tools or databases.
 - Otherwise, they diverge due to partial or hallucinated knowledge.

Agent checking the work of other agents



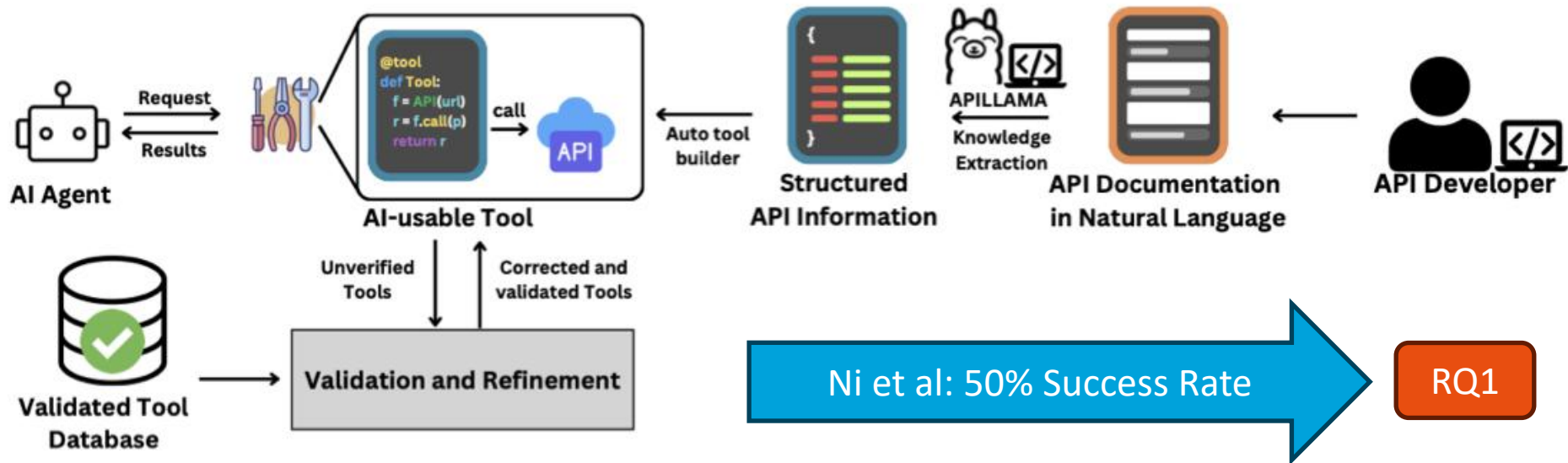
Example: our recent student projects



Maastricht University

LLM Connecting Arbitrary APIs

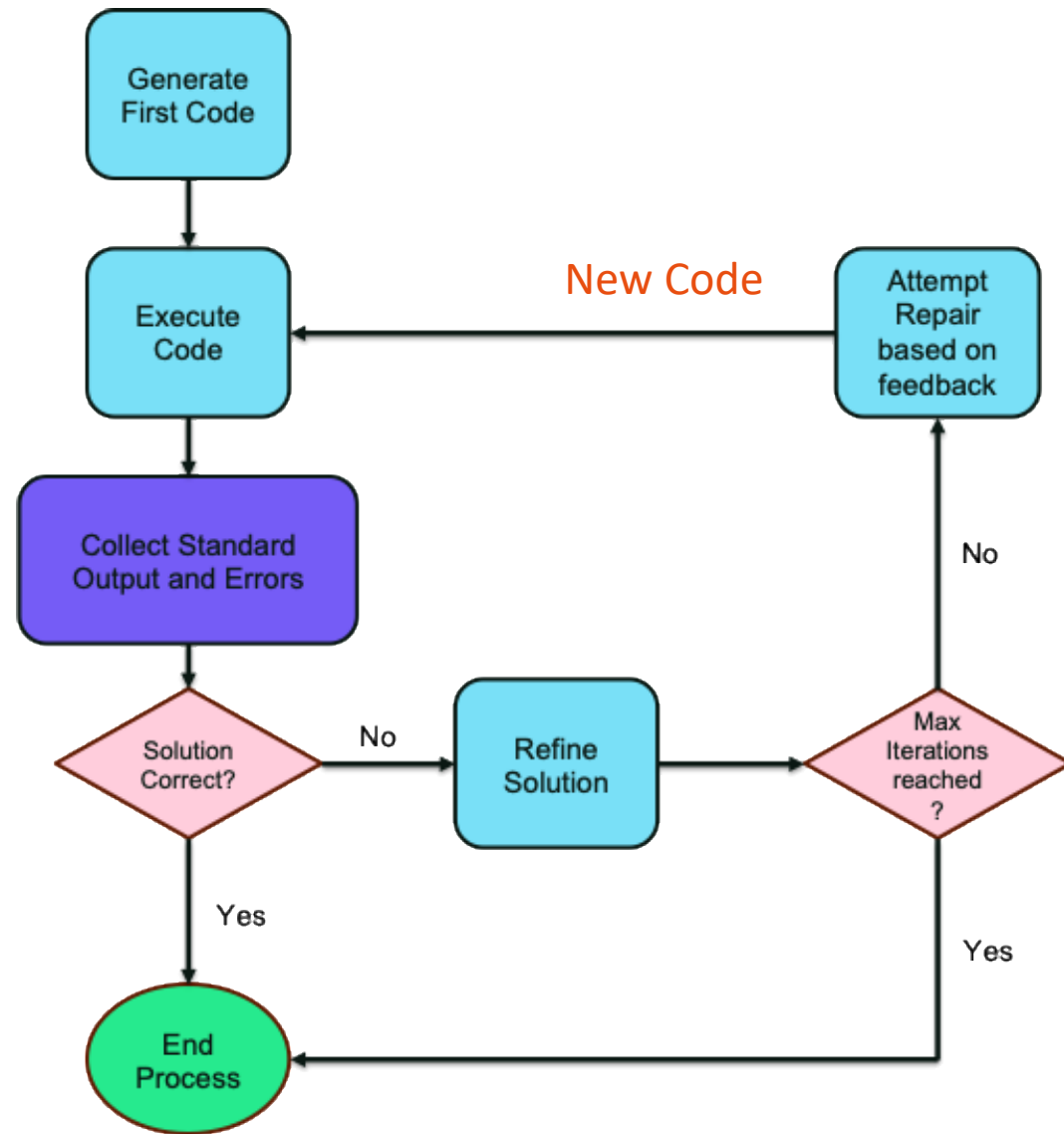
- How to create tools (agents) for LLM
- Ni et al. (2025): 167 APIList.Com REST-Apis



MSc Thesis of Jan Bergerhoff: *Higher Productivity in the Toolfactory Connecting LLMs and APIs*, DACS, 2025

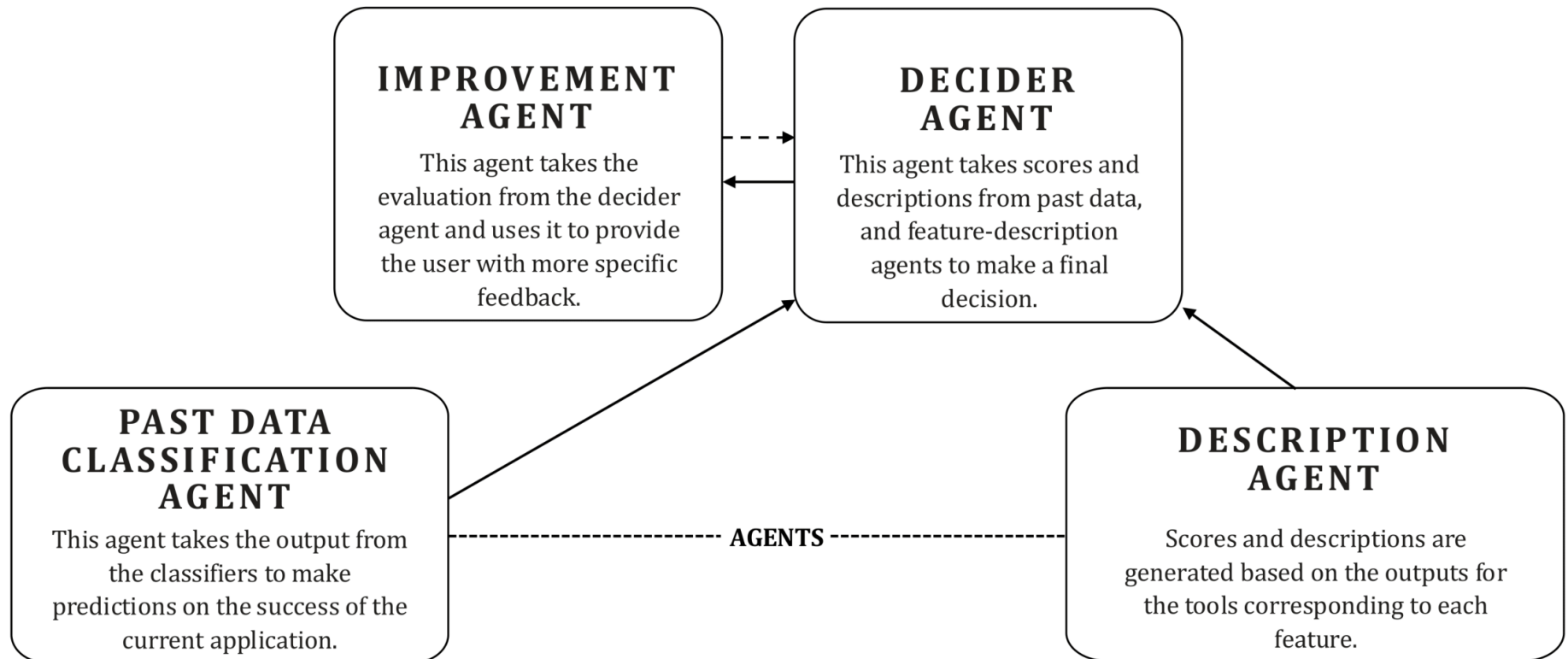
Augmentation: LLM Tool Self-Correction

- Iterative automatic debugging of the generated code
- Performance in correct API extraction improved to 89.2%

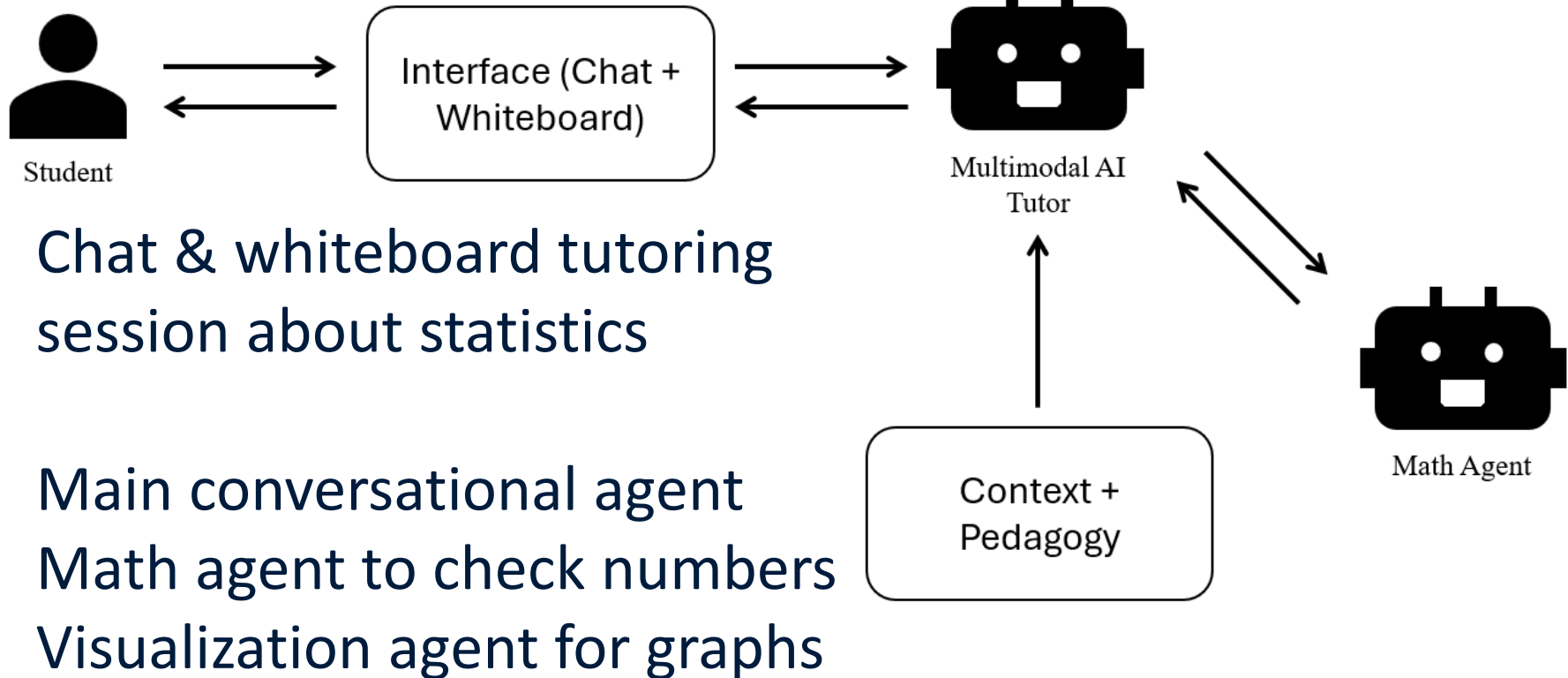


MSc Thesis of Jan Bergerhoff: *Higher Productivity in the Toolfactory Connecting LLMs and APIs*, DACS, 2025

Example: multi-agent system for supporting preparation of grant applications



Multimodal tutoring agent



Agent interoperability and standards



Maastricht University

Agent orchestration frameworks



n8n



LangChain

The platform for reliable agents.



LangGraph

Frameworks for multi-agent systems

- Mistral Agents API
- LangGraph
- OpenAI Agents SDK
- Microsoft Agent Framework
- Google Agent Development Kit
- Amazon Bedrock's AI Agent framework
- N8n – visual multiagent design and orchestration
- ...

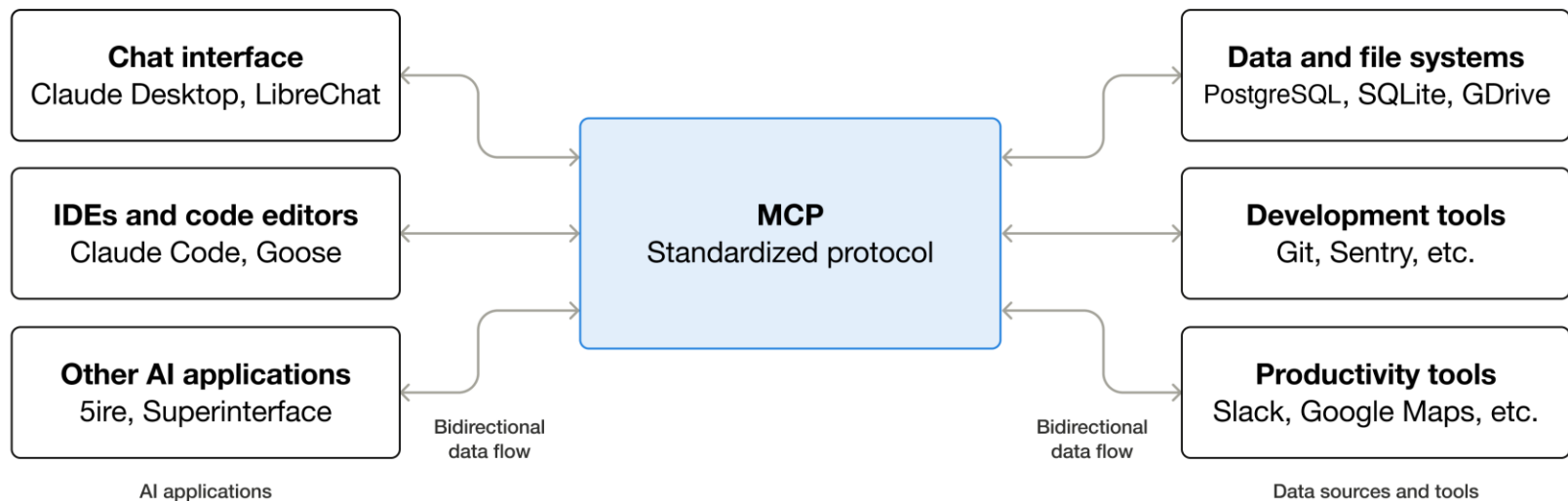
See also https://www.youtube.com/live/X_9qIYeu-wc

Agent communication protocols (standards)

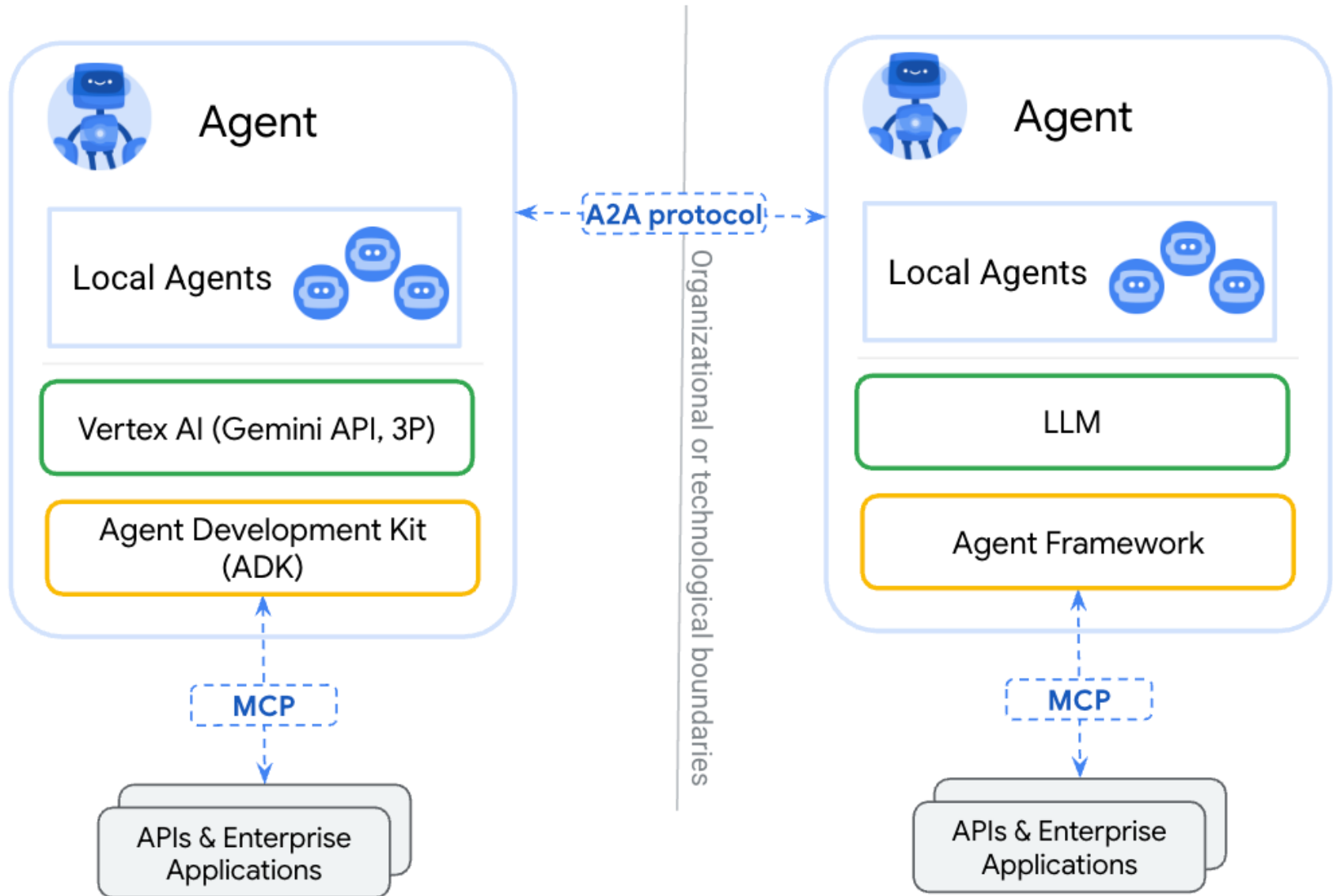
- **A2A – Agent system orchestration**
 - Agent properties, agent system config
- ANP - Agent Network Protocol
 - Agent network identities
- ACP – Agent Communication Protocol
 - Messaging format
- **MCP – Model Context Protocol**
 - Defines common tools used by agents

Model Context Protocol (MCP)

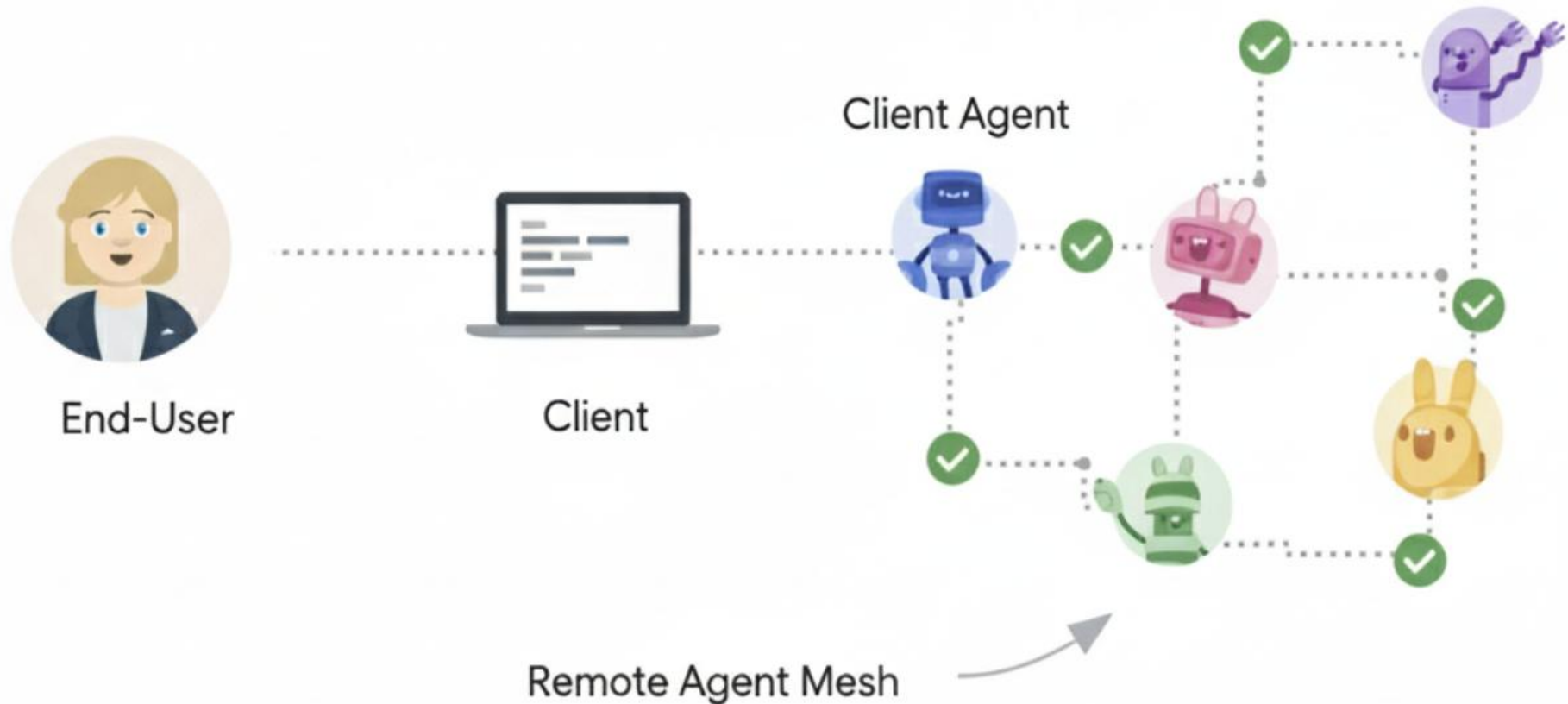
- MCP defines how LLM interface with **tools**
- Search internet/database
- Access calendar/email of the user
- Orchestrate workflows



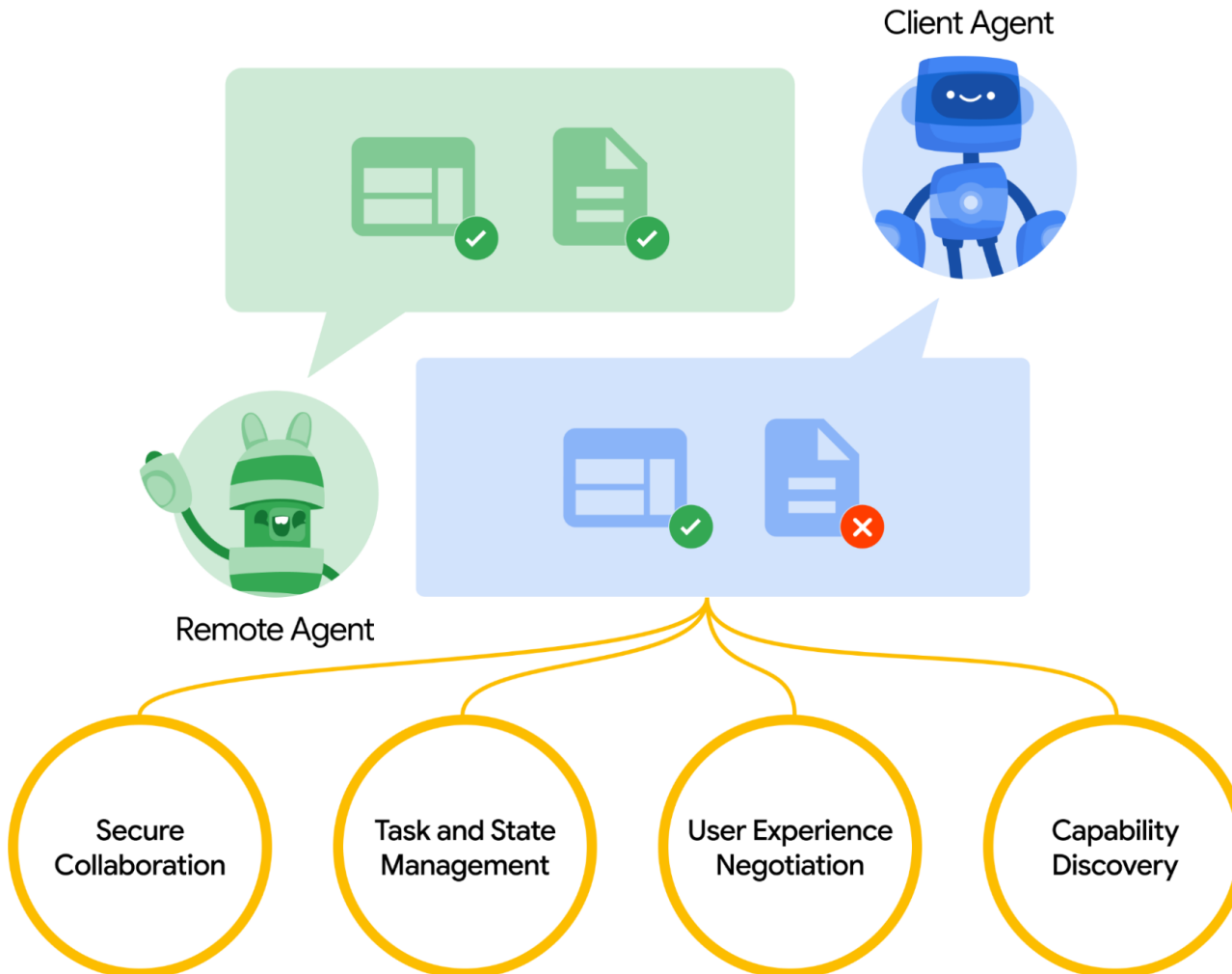
Agent2agent and Model Context Protocol



Agent2Agent (a2a) protocol



Agent2Agent (a2a) protocol



<https://github.com/a2aproject/A2A>

A2a Agent card

```
{
  "name": "Car Rental Agent",
  "description": "Helps book car rental",
  "url": "http://localhost:10105/",   "version": "1.0.0",
  "capabilities": { "streaming": true, "pushNotifications": true, "stateTransitionHistory": false},
  "defaultInputModes": ["text", "text/plain"],
  "defaultOutputModes": [ "text", "text/plain" ],
  "skills": [
    {
      "id": "book_cars",
      "name": "Book Car Rental",
      "description": "Helps with booking car rental given a criteria",
      "tags": ["Book cars"],
      "examples": [
        "Book a rental car in London, starting on June 20, 2025, and ending on July 10, 2025"
      ]
    }
  ]
}
```

<https://github.com/a2aproject/>



Agent Development Kit

Conceptual Code: Coordinator using LLM Transfer

```
from google.adk.agents import LlmAgent
```

```
billing_agent = LlmAgent(name="Billing", description="Handles billing inquiries.")
```

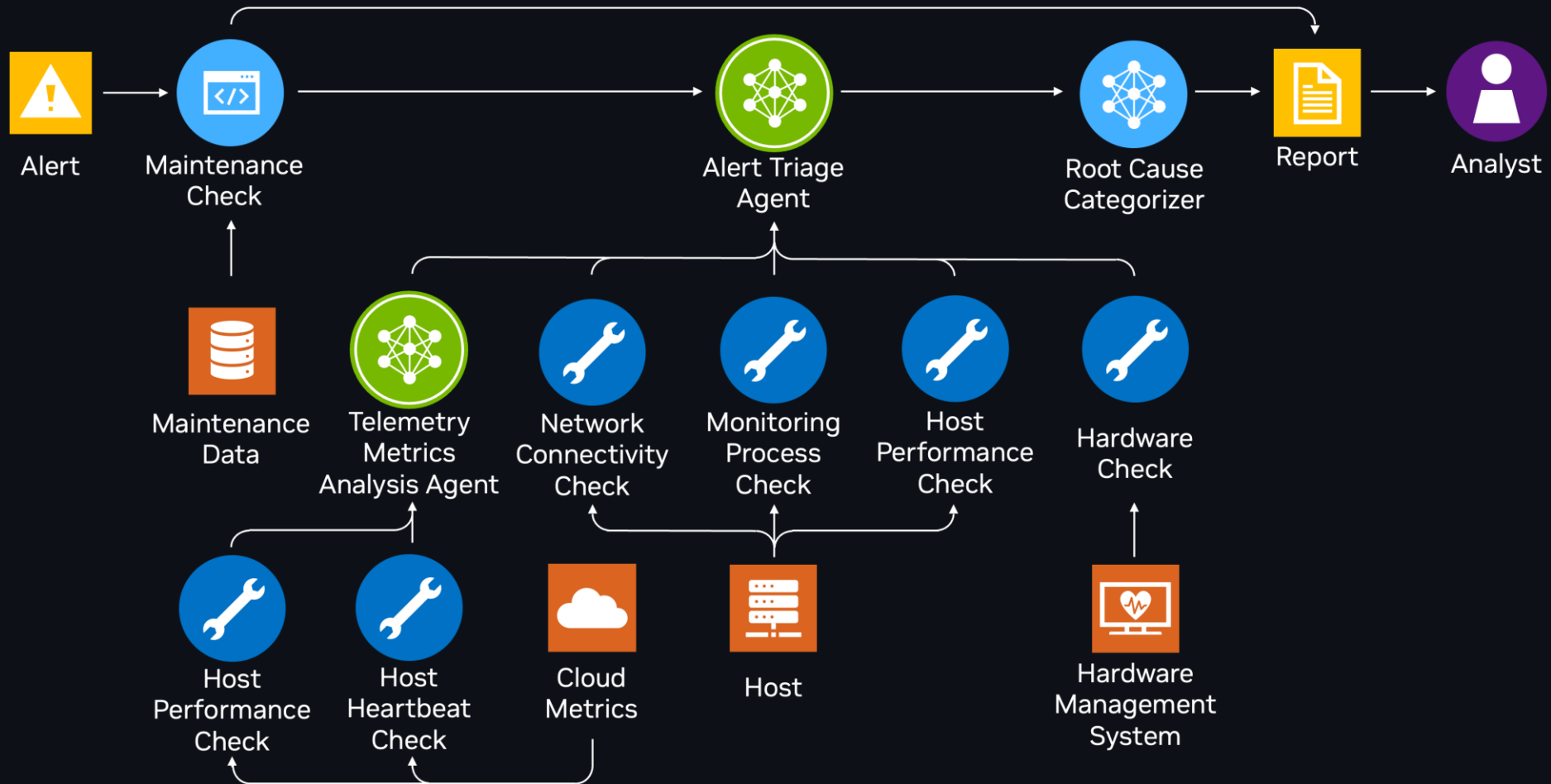
```
support_agent = LlmAgent(name="Support", description="Handles technical support requests.")
```

```
coordinator = LlmAgent(  
    name="HelpDeskCoordinator",  
    model="gemini-2.0-flash",  
    instruction="Route user requests: Use Billing agent for payment issues, Support agent for  
                technical problems.",  
    description="Main help desk router.",  
    sub_agents=[billing_agent, support_agent]  
)
```

```
# "My payment failed" -> LLM should call transfer_to_agent(agent_name='Billing')
```

```
# "I can't log in" -> LLM should call transfer_to_agent(agent_name='Support')
```

NVIDIA NeMo Agent Toolkit: Alert triage



NVIDIA NeMo Agent Toolkit

- NVIDIA's **nat** framework
- Supports MCP tools
- Agent is defined by a configuration file
- Agent workflow consists of, for example, python scripts and LLMs
- Naturally, we can use LLM to create all scripts and configs of an agent

Example: SAP Joule Agents



Explore Joule Agents by business function



Finance Agents



Supply Chain Agents



Human Resource Agents



Procurement Agents



Customer Experience Agents



Transformation Management Agents



Industries

SAP – a large German software company; a market leader in enterprise applications, corporate data management, and business planning.

Example: SAP Joule Agents



JOULE FINANCE AGENT

Receipt Analysis Agent

Eliminate manual entry and paper clutter with Expenselt's Receipt Analysis Agent that converts receipts into complete expense entries using historical data and contextual clues to fill in missing details like vendor, location, or expense type.

[View use case](#)

JOULE FINANCE AGENT

Dispute Resolution Agent

Close cases quickly and accurately with a Dispute Resolution Agent that works across functions to analyze, validate, and resolve disputes.

[View use case](#)

JOULE FINANCE AGENT

Accounts Receivable Agent

Reduce uncollectable write-offs with an Accounts Receivable Agent that analyzes overdue receivables and follows up with customers.

[View use case](#)

JOULE FINANCE AGENT

Expense Report Validation Agent

Simplify the reporting process by using the Expense Report Validation Agent to complete and submit expense reports and help resolve alerts for identifying and fixing potentially missing or incomplete expense details.

[View use case](#)

JOULE FINANCE AGENT

Meeting Location Planner Agent

Simplify group travel planning with the Meeting Location Planner Agent that presents locations for off-sites based on optimized costs, dates, flights, hotels, and venue selection—saving time and budget, and making travel seamless for teams.

[View use case](#)

JOULE FINANCE AGENT

Booking Agent

Streamline bookings and increase compliance with the Booking Agent, which delivers personalized flight and hotel recommendations based on traveler preferences, policies, and budget constraints.

[View use case](#)

JOULE FINANCE AGENT

International Trade Classification Agent

Ensure compliance with an International Trade

JOULE FINANCE AGENT

Accounting Accruals Agent

Reduce manual work, improve accuracy, and free accountants to focus on higher-value tasks with an

JOULE FINANCE AGENT

Cash Management Agent

Save up to 70% of time spent on manual reconciliation with a Cash Management Agent that

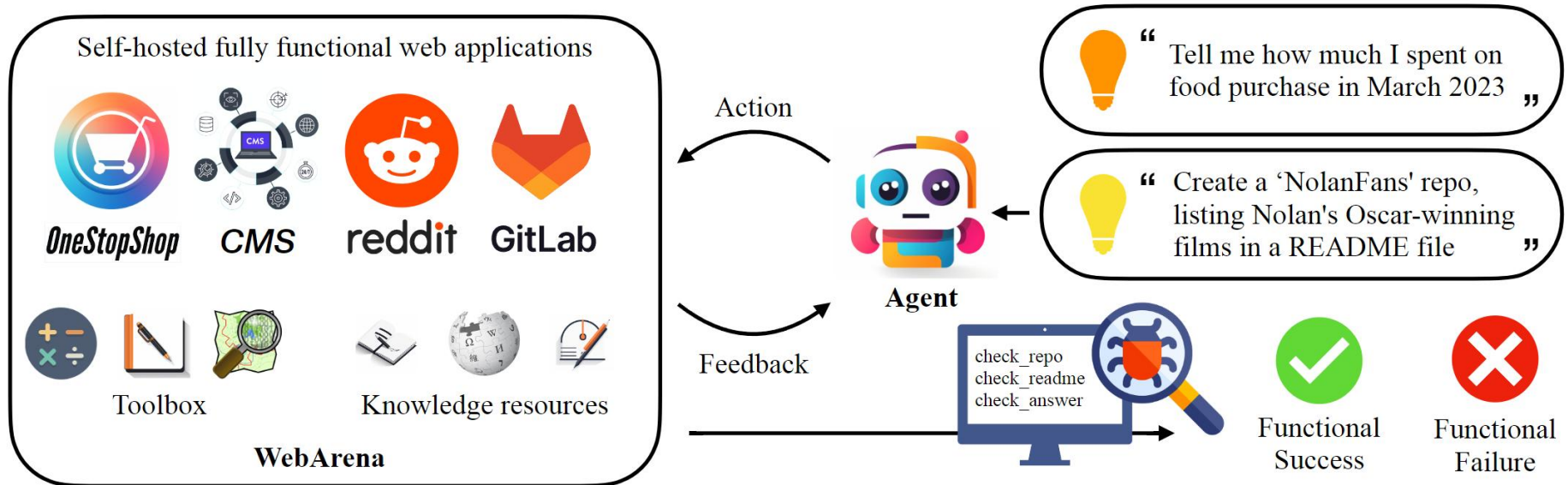


Multiagent platforms and arenas



Maastricht University

Webarena

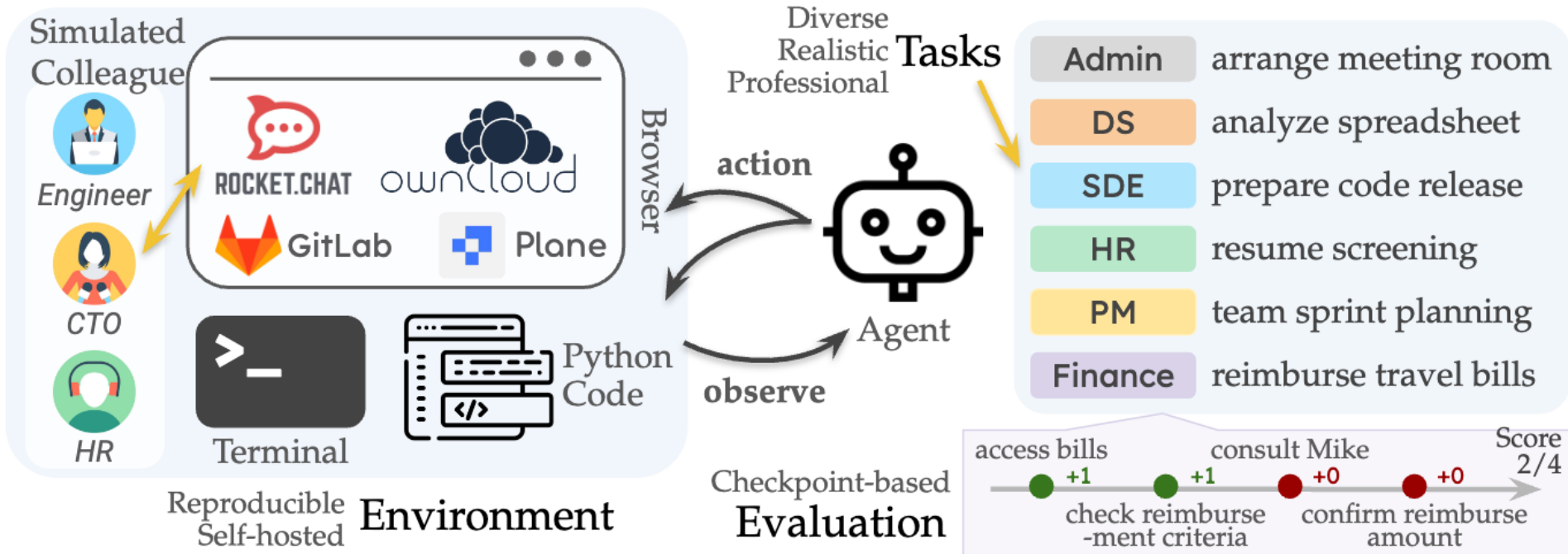


- Leaderboard: collection of typical tasks that a human would do with a browser
- Current winner success rate 68%

MultiAgentBench

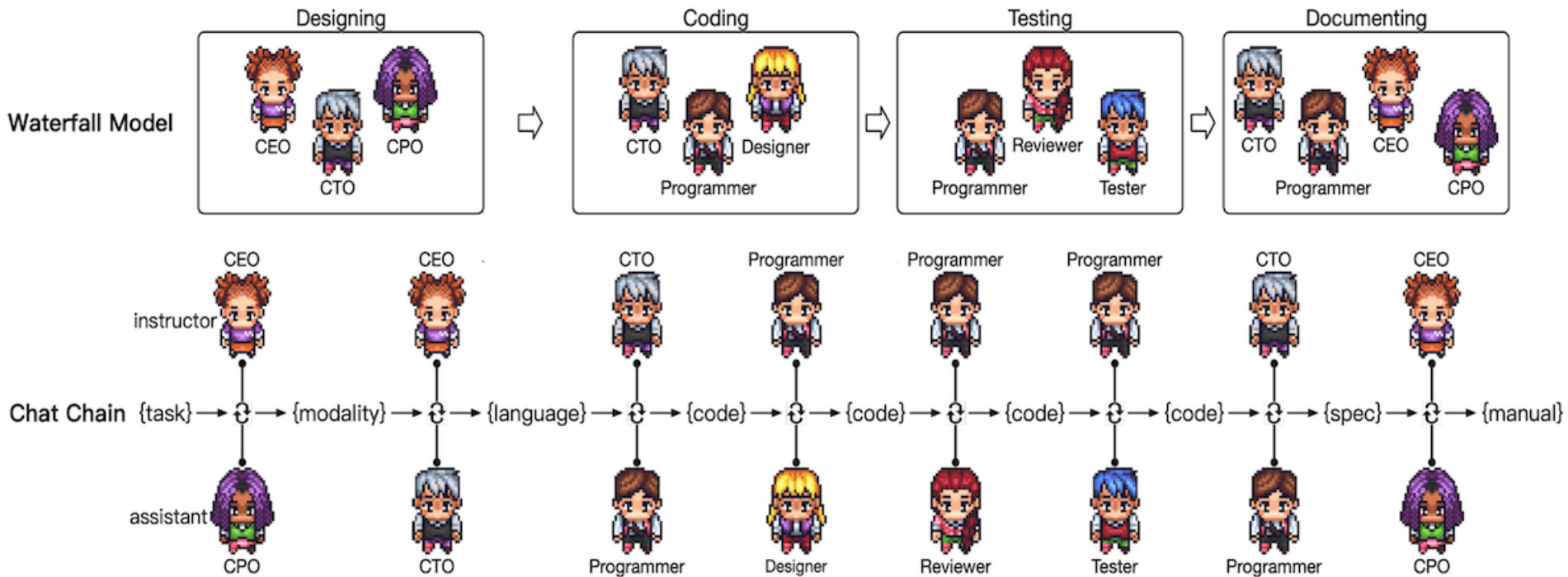


TheAgentCompany



- A virtual software company with agents as workers
- 175 tasks to solve (The current leader solves 43%)

Agentic Design Patterns: Multi-Agent Collaboration



Proposed ChatDev architecture. Image adapted from "Communicative Agents for Software Development," Qian et al. (2023).

Multiagent modeling and optimization



Maastricht University

LLM MAS formalization

In multi-agent reinforcement learning (MARL), the environment is a **stochastic game**:

$$\mathcal{G} = \langle \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N \rangle$$

- State space $\mathcal{S}$ **IS A CONTINUOUS SPACE**
- Action spaces $\mathcal{A}_i$ for each agent i **ARE CONTINUOUS**
- Transition function $P(s' \mid s, a_1, \dots, a_N)$
- Rewards $R_i(s, a_1, \dots, a_N)$ (can be cooperative, competitive, or mixed).

Each agent i has its own policy $\pi_i(a_i \mid s)$, often implemented by an LLM with role-specific prompting.

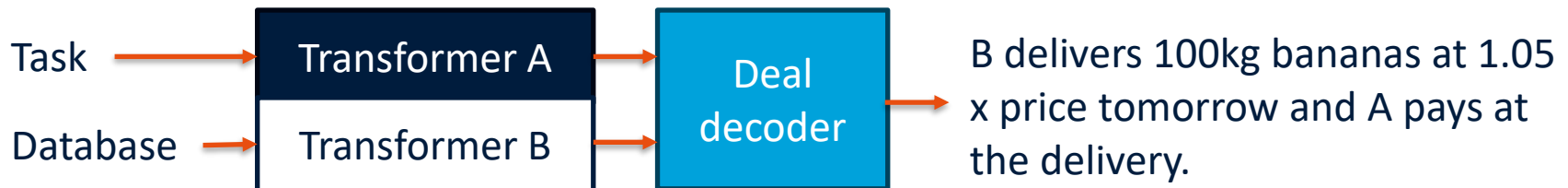
LLM MAS as negotiation v.s. generation



Negotiation

A: Hey, can you sell me 100kg bananas?
B: Now I only have 50kg
A: Can you get me 100kg?
B: Yes, but it costs 5% more.
A: Okay, when can you deliver?
B: Tomorrow.
A: Agreed, I pay at the delivery.
B: Agreed.

Joint generation of the outcome (without conversation)

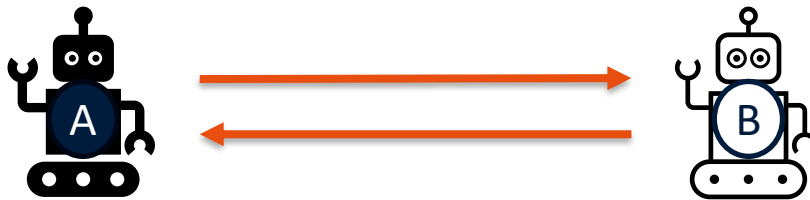


LLM-MA optimization

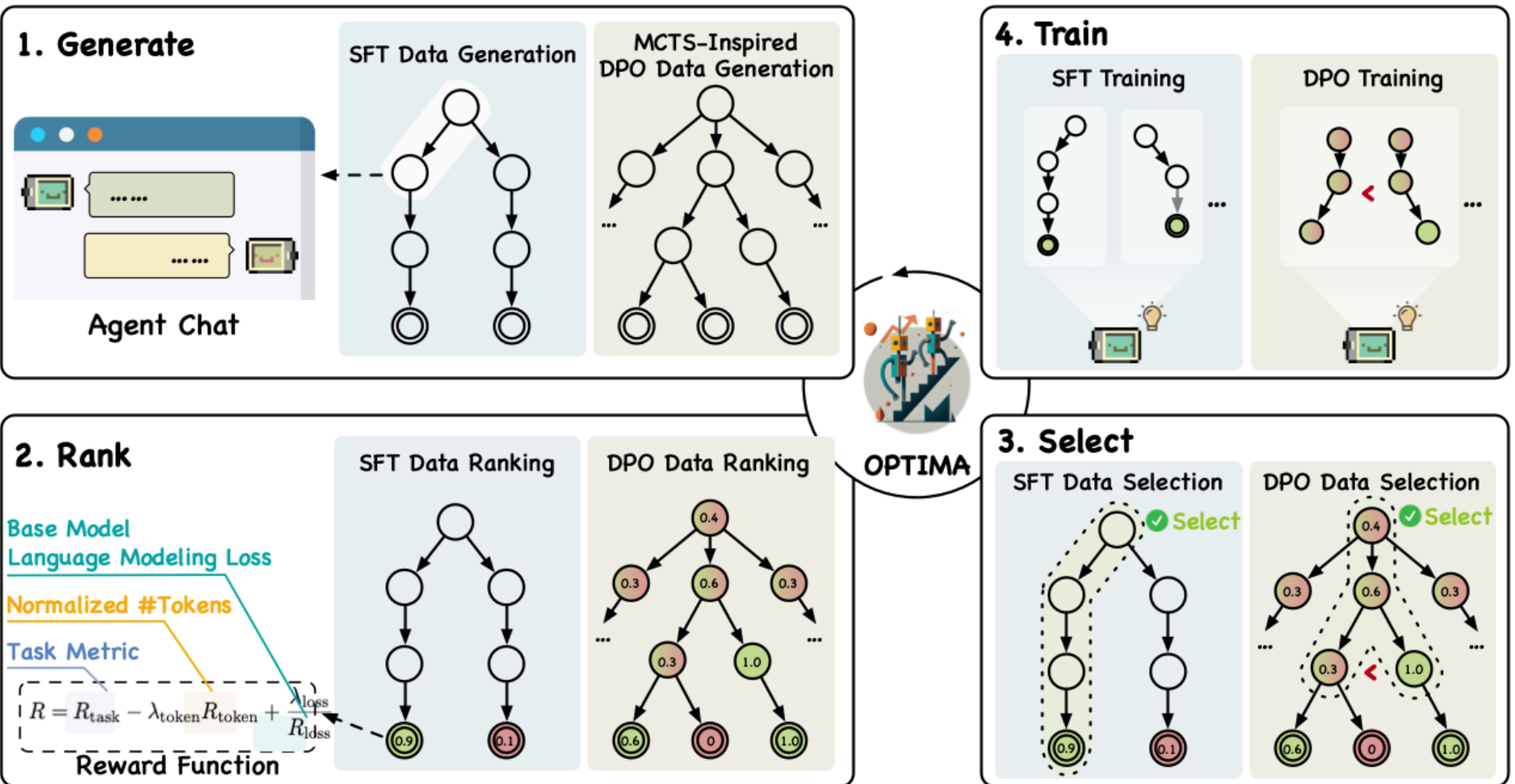
$$\hat{B}_i = \operatorname{argmax}_k P(B_{ik} | \text{Policy}_B, \text{ToM}_A, A_{i-1}, B_{i-1}, A_{i-2} \dots)$$

$$\hat{A}_i = \operatorname{argmax}_k P(A_{ik} | \text{Policy}_A, \text{ToM}_B, B_i, A_{i-1}, B_{i-1} \dots)$$

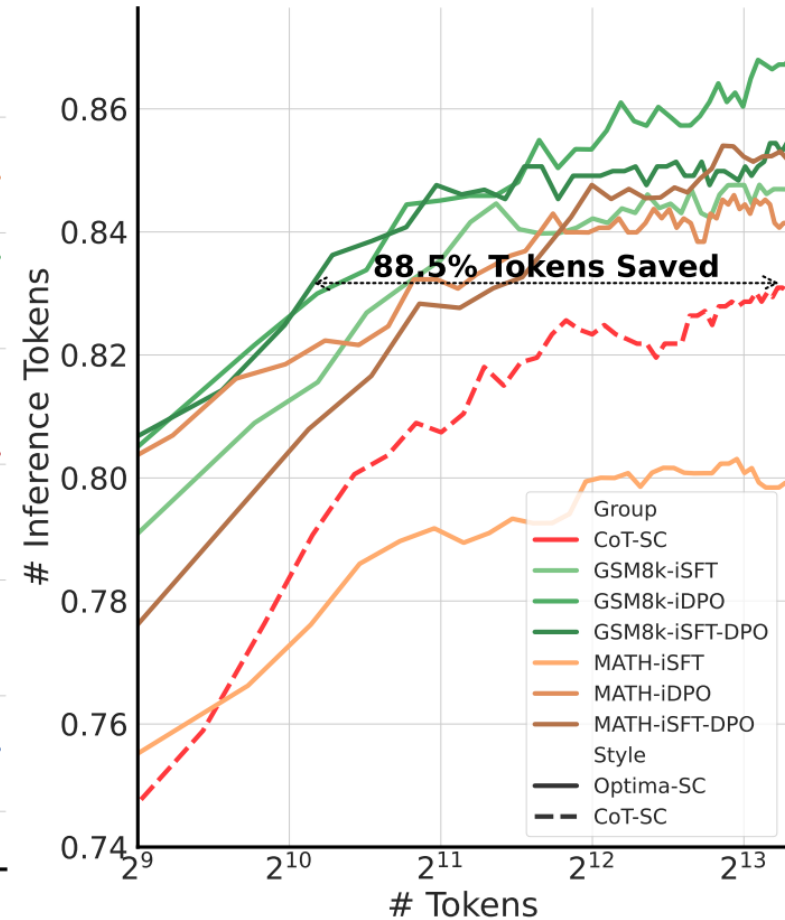
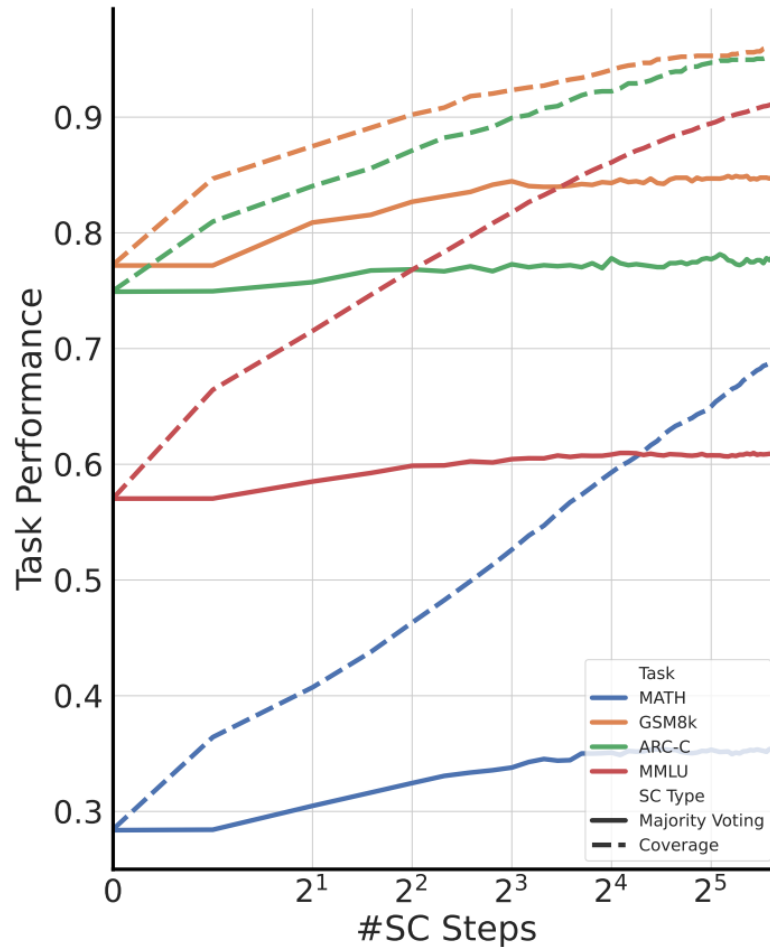
- ToM_A Theory of Mind about A (*a prompt*)
- Policy_B Policy of B (*a prompt*)



Example: optimize a2a communication



Example: optimize a2a communication



Example: A Customer Support Agent

Imagine an AI agent that can handle customer support queries for a company's products. It should answer questions, resolve issues, and know when to hand off to a human.



Maastricht University

Scenario Customer Support Agent

Key components an agent like this would need:

- **Knowledge Base Access (RAG):** It should retrieve relevant FAQs, manuals, or past ticket resolutions from the company's knowledge base to get accurate information. For instance, if a customer asks "How do I reset my device?", the agent searches the help docs for "reset device" steps and uses that text in its reply.
- **Customer Data Tools:** It might use APIs to pull the customer's account or order info. E.g., *Tool:* getOrderStatus(customer_id) if someone asks about an order, or lookupWarranty(product_serial) if it needs to check coverage. This ensures responses are personalized and correct for that specific user.
- **Interactive Planning:** The agent may have to gather information from the user in a logical sequence (like a human agent would). It plans a dialog flow: greet -> ask for account ID -> ask for issue details -> provide solution or troubleshoot step-by-step. This involves **planning the conversation** and remembering the user's answers (context carrying) across turns.
- **Reflection & Handoff:** The agent should recognize if it's unable to solve the issue (perhaps through a reflection step analyzing its own answers). If it's stuck or the user is unhappy, the **plan** might include: "Step X: escalate to human support". This is an important *safety/ethical feature*: the agent knows its limits.



How the Agent Operates

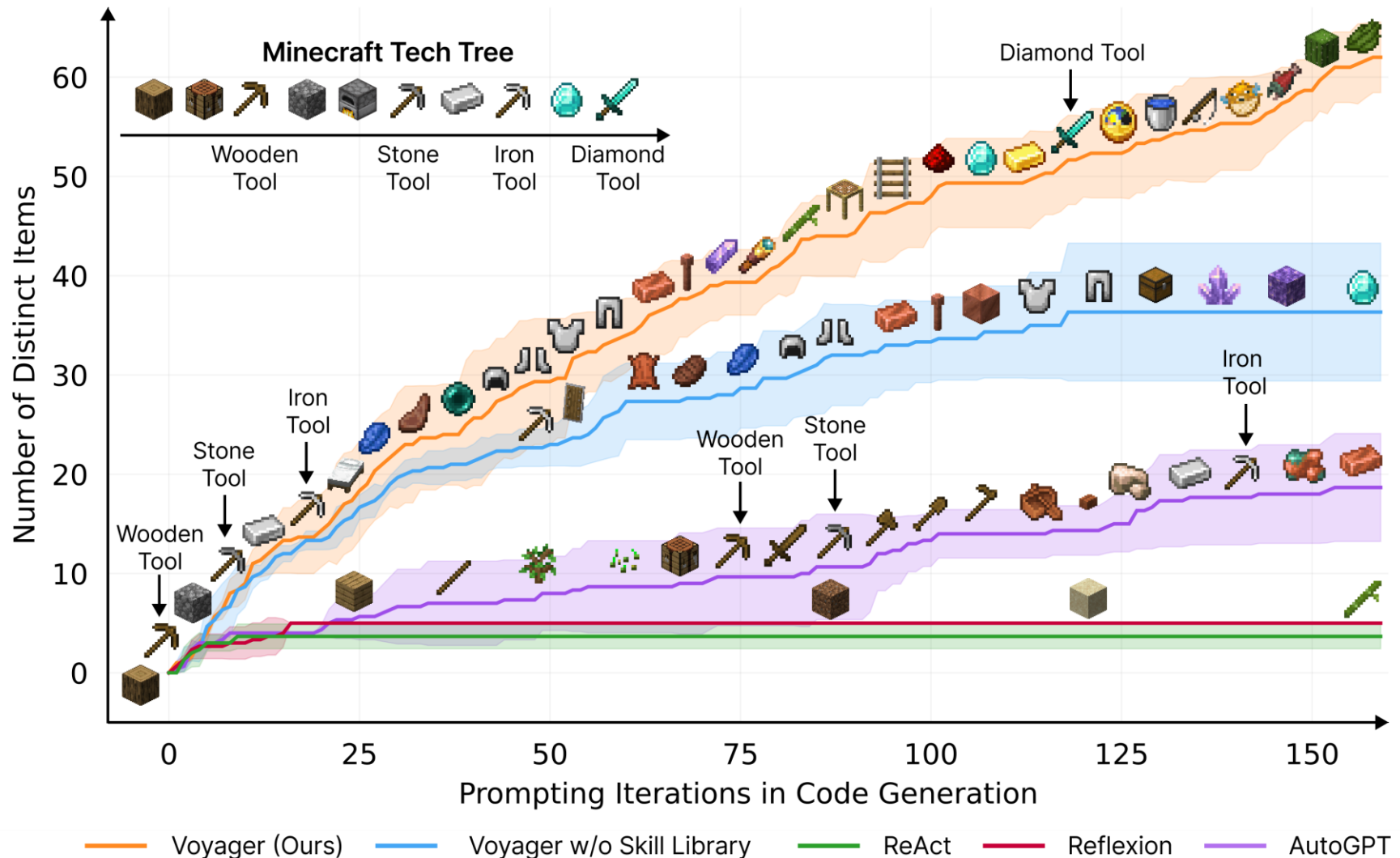
Suppose a user says, “My gadget isn’t turning on, what should I do?” An agentic support bot might do the following:

- **Retrieve Knowledge:** Find any “device won’t turn on” troubleshooting guides (via RAG it finds a relevant support article).
- **Plan Dialogue:** Decide to ask a clarifying question – maybe “Can I know which model you have?” because the steps might differ by model.
- **User responds** with model, agent then maybe **uses a tool** to check if that device is registered or under warranty.
- **Troubleshooting:** Agent uses the knowledge base info to guide the user: “Okay, let’s try a few steps: First, hold the power button for 10 seconds... Did it light up?” It might go step by step, adjusting based on the user’s replies (this adaptability is a form of *reactive planning*).
- **Reflection:** After providing a solution, agent may double-check if it covered all common fixes (“Did I address all possible causes?”). If not solved, it might loop through another set of steps or say “I’m going to transfer you to a specialist.”
- **Outcome:** Either the issue is resolved with agent guidance, or it hands over to a human agent with a summary of what was tried.

Some Frameworks for LLM Agents

- **LangChain, LlamaIndex:** libraries to build agents with memory and tool use.
- **AutoGPT, BabyAGI:** autonomous agents that decompose high-level goals into sub-tasks, execute them iteratively, and adapt based on results.
- **Voyager:** an embodied LLM agent that learns to play Minecraft by writing and executing code.
- **Cognitive Architectures:** combining LLMs with symbolic planners, knowledge graphs, or reinforcement learners (e.g., ACT-R inspired setups).

Example: performance measurement



Questions?



Maastricht University

Exercise 1: Multi-Agent Interaction Patterns

Review the section on Multi-Agent Architectures outlining different design patterns for agent collaboration.

- Describe the difference between a **Collaborative Orchestration** setup and a **Competitive Debate** setup in a multi-agent system. Provide one advantage of each approach regarding task decomposition and error correction.
- We illustrate (Agent interaction scene) various structures: Ordered, Unordered, Parallel, and Hierarchical. In a complex task like generating, formatting, and verifying a legal document, which of these structures is most appropriate and why?
- What is the fundamental difference between an agent optimizing actions in a joint embedding space versus agents using natural language negotiation? Explain why black-box embedding-space optimization might be problematic in enterprise scenarios requiring a "human in the loop".

Exercise 2: Agent Protocols and System Interoperability

Explore the communication standards that allow disparate agents and tools to interact, specifically focusing on MCP and A2A.

- Explain the purpose of the **Model Context Protocol (MCP)**. How does MCP bridge the gap between a chat interface (like Claude Desktop) and external local/remote data sources (like GitHub or PostgreSQL)?
- The **Agent2Agent (A2A)** protocol is designed to facilitate communication across organizational boundaries. According to the A2A Agent Card JSON schema provided in the lecture, what specific metadata must an agent expose to allow another agent (like a HelpDeskCoordinator) to discover and route tasks to it?
- In a scenario where a user asks "My payment failed," write a brief conceptual python script (using an LLM Agent framework) that intercepts this query, evaluates it against the capabilities of two sub-agents ("Billing" and "Support"), and routes it to the correct agent using tool calling.

Exercise 3: Evaluating Multi-Agent Systems and Theory of Mind (ToM)

Review how multi-agent systems are evaluated in complex environments and how agents model the behavior of other agents.

- Define **Theory of Mind (ToM)** in the context of multi-agent negotiation or gaming (e.g., the Rock-Paper-Scissors or Minecraft examples). How does a ToM module allow an agent to dynamically adjust its strategy based on the interaction history of an opponent?
- The lecture introduces **The Agent Company**, an evaluation benchmark. Why is evaluating a multi-agent system in a simulated software environment (with simulated colleagues like a CTO, HR, and Engineer) more robust than testing agents purely on static QA datasets like HotPotQA?
- Identify three major challenges inherent in scaling Multi-Agent Systems, specifically addressing **Communication Overhead**, **Consensus/Conflict Resolution**, and **Trust/Safety (Echo Chambers)**. Provide a brief mitigation strategy for each challenge.

Discussion



On Why Machines Should Talk: Is Language a Bottleneck, or the Record?

Agents in a multi-agent system mostly talk to each other in English: slow to generate, expensive, and ambiguous. They could instead be coupled directly and settle their business in a shared latent space, with no words at all (slide 42).

Why do we insist that machines talk? If language is a bottleneck we invented for ourselves, are we imposing it on them out of habit, or because a negotiation nobody can read is not a negotiation anyone can be held to?

Human institutions run on readable language for a reason: contracts, minutes, testimony. If agents stop talking, what replaces the record? And is oversight something that can survive being made efficient?

On the Echo Chamber of Clones: Can a Model Audit Itself in Disguise?

To catch errors we add a critic agent to check the generator, or let two agents debate. In most deployments the critic and the generator are the same model with a different prompt: the same training data, the same blind spots, the same confident mistakes.

Can a model audit itself by being told it is someone else? Is a debate between two copies a test of the truth, or an elaborate way for one model to agree with itself?

Science and law rely on adversaries who are genuinely different. What would genuine difference mean for agents, and what would it cost? And how would we ever notice if our critics were simply colluding with our generators on a shared mistake?

On Theory of Mind: Is There Another Mind in the Model, or Only Its Transcript?

In games and bargaining, an agent is given a Theory of Mind module: a prompt in which it hypothesises what its opponent knows and will do next. It works; the agent adapts.

When the model writes “my opponent is countering my last move”, is there a representation of another mind anywhere in the system, or only the most probable continuation of a game transcript? Is there a difference, and what observation would reveal it?

Humans attribute minds to each other without ever seeing one. If strategic empathy in humans is also inference from behaviour, what is the model missing, and could it ever adapt to an opponent whose strategy no transcript has ever contained?